Shortest Path 101

SHSID CPC

Methods

Unweighted: BFS
Non-negative weighted: Dijkstra
With negative: **Bellman-Ford**, especially if you want to detect negative loops
Want all pairs of shortest distances: **Floyd**

No weights BFS

```
d[v] ← ∞  ∀v ∈ V; d[v_0] ← 0
queue q; q.push(v_0)
while !q.empty() do
    Vertex cur ← q.pop()
    for Nachbar v von cur do
        if d[cur] + 1 < d[v] then
            q.push(v)
            d[v] ← d[cur] + 1
        end
    end
end
```

USACO 2016 February Contest, Silver
Problem 3. Milk Pails

Return to Problem List

Contest has ended.

Analysis mode

English (en)

Farmer John has received an order for exactly M units of milk ($1 \le M \le 200$) that he needs to fill right away. Unfortunately, his fancy milking machine has just become broken, and all he has are two milk pails of integer sizes X and Y ($1 \le X, Y \le 100$) with which he can measure milk. Both pails are initially empty. Using these two pails, he can perform up to K of the following types of operations ($1 \le K \le 100$):

- He can fill either pail completely to the top.

- He can empty either pail.

- He can pour the contents of one pail into the other, stopping when the former becomes empty or the latter becomes full (whichever of these happens first).

Although FJ realizes he may not be able to end up with exactly M total units of milk in the two pails, please help him compute the minimum amount of error between M and the total amount of milk in the two pails. That is, please compute the minimum value of $|M - M'|$ such that FJ can construct M' units of milk collectively between the two pails.

INPUT FORMAT (file pails.in):

The first, and only line of input, contains X, Y, K, and M.

OUTPUT FORMAT (file pails.out):

Output the smallest distance from M to an amount of milk FJ can produce.

SAMPLE INPUT:

```
14 50 2 32
```

SAMPLE OUTPUT:

```
18
```

In two steps FJ can be left with the following quanities in his pails

```
(0, 0) = 0 units
(14, 0) = 14 units
(0, 50) = 50 units
(0, 14) = 14 units
(14, 36) = 50 units
(14, 50) = 64 units
```

The closest we can come to 32 units is 14 for a difference of 18. Note that it would require an extra step to pour out the first pail to end up with (0, 36).

Problem credits: Brian Dean

USACO 2016 Feb Silver P3: Milk Pails

Feels just like try, check, and no revise (it doesn’t seem like there are many cases...)

And... yes, indeed, there aren't a lot of cases, so doing a BFS over every single possible configuration of milk volumes works!

Time complexity: O(XY)

USACO 2020 December Contest, Gold
Problem 1. Replication

Return to Problem List

Contest has ended.

Log in to allow submissions in analysis mode

English (en)

The ill-fated result of watching too many "do it yourself" engineering videos on the web, Farmer John has accidentally released a self-replicating robot on his farm!

The farm can be represented by an $N \times N$ grid ($3 \le N \le 1000$) where each grid cell is either empty or filled with rock, and all border squares are filled with rock. Some non-rock cells are designated as possible starting locations for the robot.

Farmer John initially places the robot at one of the possible starting positions. In every hour that follows, all copies of the robot move in one coordinated mass in the same direction, either north, south, east, or west. After every D hours ($1 \le D \le 10^9$), every copy of the robot replicates --- a robot at cell (x, y) that replicates creates new copies in cells $(x + 1, y)$, $(x - 1, y)$, $(x, y + 1)$, and $(x, y - 1)$; the original robot remains at (x, y). Over time, multiple robots might come to occupy the same cell.

If moving or replicating would cause any of the robots to move into a rock, then all robots shut down immediately. Note that this implies that the robots must eventually shut down, due to the border of the farm being rock.

Help the cows figure out the number of empty squares that could potentially at some point in time hold a robot.

INPUT FORMAT (input arrives from the terminal / stdin):

The first line contains two space-separated integers N and D. The next N lines of input each contain N characters. Each character is one of '.', 'S', or '#'. '.' and 'S' both represent empty cells, with 'S' denoting a possible starting position for the robot. '#' denotes a rock.

All characters in the first and last row and first and last column are '#'.

OUTPUT FORMAT (print output to the terminal / stdout):

An integer counting the number of cells that could at some point in time hold a robot.

SAMPLE INPUT:

```
10 1
#########
#.......#
#S......#
#.......#
#########
#S....S..#
#########
#########
#########
#########
```

SAMPLE OUTPUT:

```
15
```

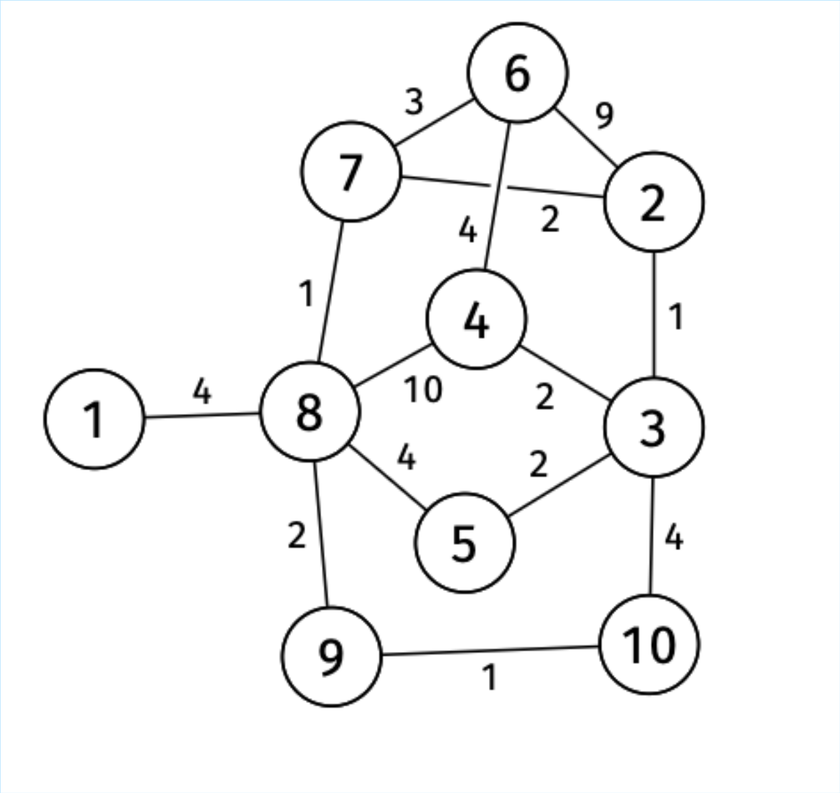
USACO 2020 Dec Gold P1: Replication

Note that the robots form a clump, which at time t is dictated by $|x+y| \leq t - 1$ before replication (with the exception of $t = 0$), where x and y are the distances to the “central robot”, or the robot we started with. Thus, only rocks that are of Manhattan distance of t or less with respect to the central bot can cause a shutdown.

Then, we can just BFS over all nodes visit-able by a central node, and from there, get which nodes could have contained a robot.

Time complexity: $O(N \wedge 2)$ with a relatively large constant factor.

Dijkstra



Algorithm 1: Shortest path based on the Dijkstra algorithm

```
for all v ≠ u such that L(v) = w(u,v) do
  L(u) = 0
  T = u
  while T≠V do
    Find v' ∉ such that ∀ v∉ T, L(v') ≤ L(v)
    T = T ∪ v'
    for all v∉ T such that v' is adjacent to v do
        if L(v)>L(v') + w(v',v) then
            L(v) = L(v') + w(v',v)
        end if
    end for
  end while
end for
```

Dijkstra

```
void dij(int st){
    memset(vis, 0, sizeof(vis));
    memset(d, 0x3f, sizeof(d));
    priority_queue<pair<long long, int>> pq;
    pq.push({0,st});
    d[st] = 0;
    while (!pq.empty()){
        int x = pq.top().second;
        pq.pop();
        if (vis[x]) continue;
        vis[x] = 1;
        for (auto [v, w]: G[x]){
            if (d[v] > d[x]+w){
                d[v] = d[x]+w;
                pq.push({-d[v], v});
            }
        }
    }
}
```

CSES Problem Set

Shortest Routes I

TASK STATISTICS

Time limit: 1.00 s **Memory limit:** 512 MB

There are n cities and m flight connections between them. Your task is to determine the length of the shortest route from Syrjälä to every city.

Input

The first input line has two integers n and m: the number of cities and flight connections. The cities are numbered $1, 2, \ldots, n$, and city 1 is Syrjälä.

After that, there are m lines describing the flight connections. Each line has three integers a, b and c: a flight begins at city a, ends at city b, and its length is c. Each flight is a one-way flight.

You can assume that it is possible to travel from Syrjälä to all other cities.

Output

Print n integers: the shortest route lengths from Syrjälä to cities $1, 2, \ldots, n$.

Constraints

- $1 \le n \le 10^5$
- $1 \le m \le 2 \cdot 10^5$
- $1 \le a, b \le n$
- $1 \le c \le 10^9$

Example

Input:

```
3 4
1 2 6
1 3 2
3 2 3
1 3 4
```

Output:

```
0 5 2
```

Shortest Routes II

TASK | STATISTICS

Time limit: 1.00 s **Memory limit:** 512 MB

There are n cities and m roads between them. Your task is to process q queries where you have to determine the length of the shortest route between two given cities.

Input

The first input line has three integers n, m and q: the number of cities, roads, and queries.

Then, there are m lines describing the roads. Each line has three integers a, b and c: there is a road between cities a and b whose length is c. All roads are two-way roads.

Finally, there are q lines describing the queries. Each line has two integers a and b: determine the length of the shortest route between cities a and b.

Output

Print the length of the shortest route for each query. If there is no route, print -1 instead.

Constraints

- $1 \leq n \leq 500$
- $1 \leq m \leq n^2$
- $1 \leq q \leq 10^5$
- $1 \leq a, b \leq n$
- $1 \leq c \leq 10^9$

Example

Input:
```
4 3 5
1 2 5
1 3 9
2 3 3
1 2
2 1
1 3
1 4
3 2
```

Output:
```
5
5
8
−1
3
```

Graph Algorithms

...

- Round Trip
- Monsters
- Shortest Routes I
- Shortest Routes II
- High Score
- Flight Discount
- Cycle Finding
- Flight Routes

...

Flight Discount

TASK | STATISTICS

Time limit: 1.00 s **Memory limit:** 512 MB

Your task is to find a minimum-price flight route from Syrjälä to Metsälä. You have one discount coupon, using which you can halve the price of any single flight during the route. However, you can only use the coupon once.

When you use the discount coupon for a flight whose price is x, its price becomes $\lfloor x/2 \rfloor$ (it is rounded down to an integer).

Input

The first input line has two integers n and m: the number of cities and flight connections. The cities are numbered $1, 2, \ldots, n$. City 1 is Syrjälä, and city n is Metsälä.

After this there are m lines describing the flights. Each line has three integers a, b, and c: a flight begins at city a, ends at city b, and its price is c. Each flight is unidirectional.

You can assume that it is always possible to get from Syrjälä to Metsälä.

Output

Print one integer: the price of the cheapest route from Syrjälä to Metsälä.

Constraints

- $2 \leq n \leq 10^5$
- $1 \leq m \leq 2 \cdot 10^5$
- $1 \leq a, b \leq n$
- $1 \leq c \leq 10^9$

Example

Input:
```
3 4
1 2 3
2 3 1
1 3 7
2 1 5
```

Output:
```
2
```

USACO 2019 December Contest, Gold
Problem 1. Milk Pumping

Return to Problem List

Contest has ended.

Log in to allow submissions in analysis mode

English (en)

Farmer John has recently purchased a new farm to expand his milk production empire. The new farm is connected to a nearby town by a network of pipes, and FJ wants to figure out the best set of these pipes to purchase for his use in pumping milk from the farm to the town.

The network of pipes is described by N junction points (endpoints of pipes), conveniently numbered $1 \ldots N$ ($2 \le N \le 1000$). Junction point 1 represents FJ's farm and junction point N is the town. There are M bi-directional pipes ($1 \le M \le 1000$), each joining a pair of junction points. The ith pipe costs c_i dollars for FJ to purchase for his use, and can support a flow rate of f_i liters of milk per second.

FJ wants to purchase a single path worth of pipes, where the endpoints of the path are junctions 1 and N. The cost of the path is the sum of the costs of the pipes along the path. The flow rate along the path is the minimum of the flow rates of the pipes along the path (since this serves as a bottleneck for the flow traveling down the path). FJ wants to maximize the flow rate of the path divided by the cost of the path. It is guaranteed that a path from 1 to N exists.

SCORING:

- Test cases 2-5 satisfy $N, M \le 100$.

INPUT FORMAT (file pump.in):

The first line of input contains N and M. Each of the following M lines describes a pipe in terms of four integers: a and b (the two different junctions connected by the pipe), c (its cost), and f (its flow rate). Cost and flow rate are both positive integers in the range $1 \ldots 1000$.

OUTPUT FORMAT (file pump.out):

Please print 10^6 times the optimal solution value, truncated to an integer (that is, rounded down to the next-lowest integer if this number is not itself an integer).

SAMPLE INPUT:

```
3 2
2 1 2 4
2 3 5 3
```

SAMPLE OUTPUT:

```
428571
```

In this example, there is only one path from 1 to N. Its flow is $\min(3, 4) = 3$ and its cost is $2 + 5 = 7$.

Problem credits: Brian Dean

USACO 2019 January Contest, Gold
Problem 3. Shortcut

Return to Problem List

Contest has ended.

Log in to allow submissions in analysis mode

English (en)

Every evening, Farmer John rings a giant bell that summons his cows to the barn for dinner. Eager to get to the barn as quickly as possible, they all follow the shortest possible route to get there.

The farm is described by a set of N fields ($1 \le N \le 10,000$), conveniently numbered $1 \ldots N$, with the barn residing in field 1. The fields are connected by a set of M bidirectional trails ($N - 1 \le M \le 50,000$). Each trail has a travel time associated with it, and there is a path from every field to the barn using some set of trails.

Field i contains c_i cows. Upon hearing the dinner bell, these cows all walk to the barn along a route that takes the minimum amount of time. If there are several routes tied for the minimum time, the cows take whichever of these is "lexicographically" smallest (i.e., they break ties between two routes by favoring the one using the lower-indexed field at the first place where the routes differ, so for example a path that visits fields 7, 3, 6, 1 would be preferable to one that visits 7, 5, 1, assuming both had the same travel time).

Farmer John is worried about the barn being far away from some fields. He adds up the travel time experienced by each cow, summed over all the cows, calling this number the total travel time. He would like to reduce this number as much as possible by adding one extra "shortcut" trail which has a travel time of T ($1 \le T \le 10,000$), from the barn (field 1) to some other field of his choosing. If a cow stumbles upon the shortcut trail while traveling along her usual path to the barn, she will take it if it gets her to the barn faster. Otherwise, a cow will follow her usual route, even if it might have been possible to use the shortcut to improve her travel time.

Please help Farmer John determine the greatest possible amount of decrease in total travel time he can achieve by adding his shortcut trail.

INPUT FORMAT (file shortcut.in):

The first line of input contains N, M, and T. The next line contains N integers $c_1 \ldots c_N$, each in the range $0 \ldots 10,000$. The next M lines each describe a trail using three integers a, b, and t, where the trail connects fields a and b and has travel time t. All travel times are in the range $1 \ldots 25,000$.

OUTPUT FORMAT (file shortcut.out):

Please output the largest possible reduction in total travel time Farmer John can achieve.

SAMPLE INPUT:

```
5 6 2
1 2 3 4 5
1 2 5
1 3 3
2 4 3
3 4 5
4 5 2
3 5 7
```

SAMPLE OUTPUT:

```
40
```

```
D[1] = 0; pq.push({ 0, 1 });
while(!pq.empty()) {
    auto [d, u] = pq.top(); pq.pop();

    if(vis[u]) continue;
    vis[u] = 1;

    for(auto &[v, w] : G[u]) {
        if(D[v] > d + w) {
            D[v] = d + w;
            par[v] = u;
            pq.push({ d + w, v });
        } else if(D[v] == d + w && u < par[v]) {
            par[v] = u;
            pq.push({ d + w, v });
        }
    }
}

vector<ll> cnt(n+1);
for(int i=1; i<=n; i++) {
    int curr = i;
    while(curr) {
        cnt[curr] += c[i];
        curr = par[curr];
    }
}

ll ans = 0;
for(int i=1; i<=n; i++)
    if(D[i] > T) ans = max(ans, (D[i] - T) * cnt[i]);
cout << ans << '\n';
return 0;
```

E. Not Escaping

time limit per test: 2 seconds
memory limit per test: 256 megabytes

Major Ram is being chased by his arch enemy Raghav. Ram must reach the top of the building to escape via helicopter. The building, however, is on fire. Ram must choose the optimal path to reach the top of the building to lose the minimum amount of health.

The building consists of n floors, each with m rooms each. Let (i, j) represent the j-th room on the i-th floor. Additionally, there are k ladders installed. The i-th ladder allows Ram to travel from (a_i, b_i) to (c_i, d_i), but **not in the other direction**. Ram also gains h_i health points if he uses the ladder i. **It is guaranteed $a_i < c_i$ for all ladders.**

If Ram is on the i-th floor, he can move either left or right. Travelling across floors, however, is treacherous. If Ram travels from (i, j) to (i, k), he loses $|j - k| \cdot x_i$ health points.

Ram enters the building at $(1, 1)$ while his helicopter is waiting at (n, m). What is the minimum amount of health Ram loses if he takes the most optimal path? Note this answer may be negative (in which case he gains health). Output "`NO ESCAPE`" if no matter what path Ram takes, he cannot escape the clutches of Raghav.

Input

The first line of input contains t ($1 \le t \le 5 \cdot 10^4$) — the number of test cases.

The first line of each test case consists of 3 integers n, m, k ($2 \le n, m \le 10^5$; $1 \le k \le 10^5$) — the number of floors, the number of rooms on each floor and the number of ladders respectively.

The second line of a test case consists of n integers $x_1, x_2, \ldots, x_n$ ($1 \le x_i \le 10^6$).

The next k lines describe the ladders. Ladder i is denoted by a_i, b_i, c_i, d_i, h_i ($1 \le a_i < c_i \le n$; $1 \le b_i, d_i \le m$; $1 \le h_i \le 10^6$) — the rooms it connects and the health points gained from using it.

It is guaranteed $a_i < c_i$ for all ladders and there is **at most** one ladder between any 2 rooms in the building.

The sum of n, the sum of m, and the sum of k over all test cases do not exceed 10^5.

Output

Output the minimum health Ram loses on the optimal path from $(1, 1)$ to (n, m). If Ram cannot escape the clutches of Raghav regardless of the path he takes, output "`NO ESCAPE`" (all uppercase, without quotes).

Example